

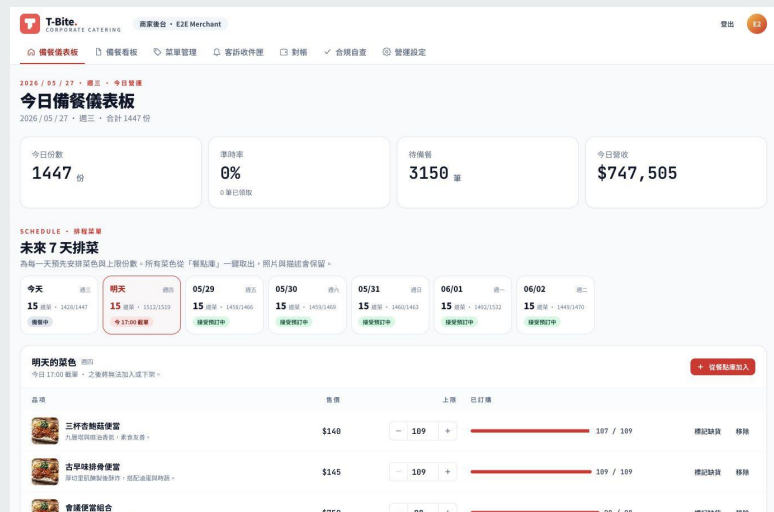
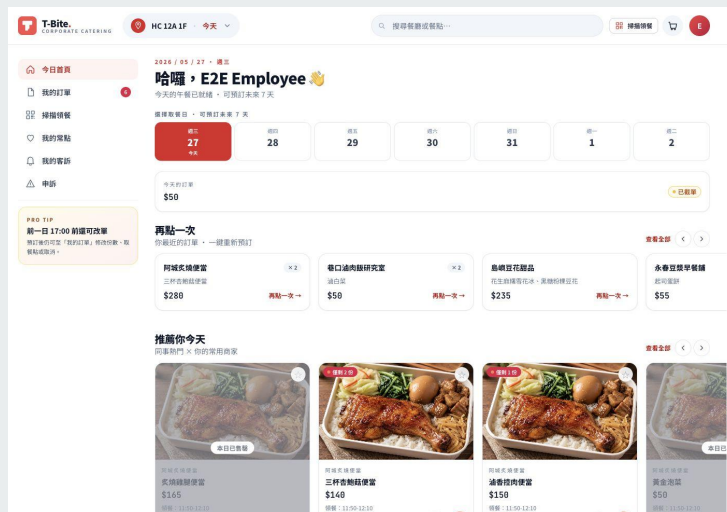
AI-Agent-Driven Enterprise Catering Platform:

From Greenfield Generation to Self-Hostable Kubernetes Validation



T-Bite.
CORPORATE CATERING

Team 10
林振可 Jacob
王重鈞
Takala
邱可茵 DW
洪軾凱 Sky



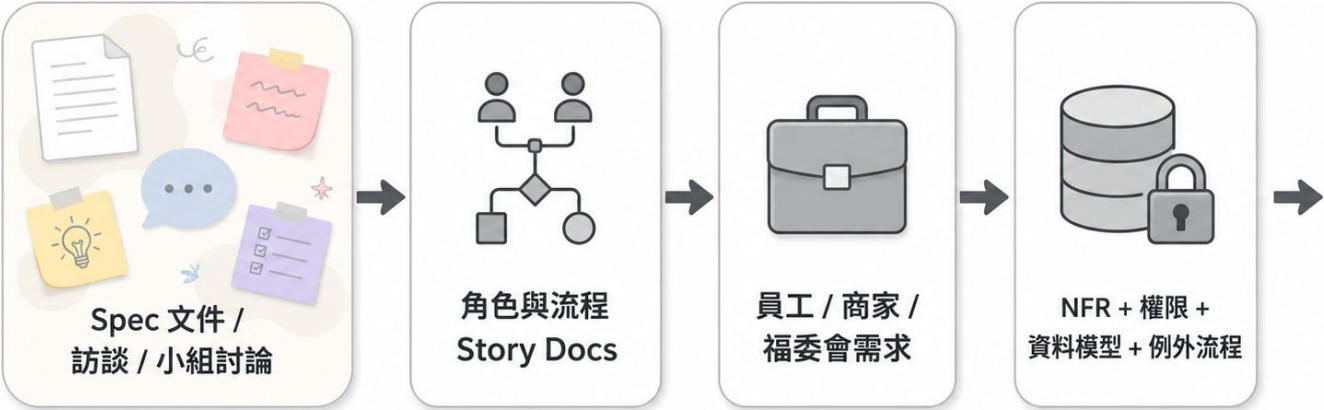
Agentic-Build/corporate-catering-system

We delivered: Product, Platform, and an harness workflow.

- | | |
|------------------------|---|
| 1. Method | How we build the whole system and workflow |
| 2. Architecture | How main system and supply system work together |
| 3. Design | How it solves enterprise catering |
| 4. Testing | How we verified reliability and scale |
| 5. Operations | CI/CD + DevOps + Observability |
| 6. Conclusion | What we delivered |

Method

Method: Requirements Transformation



企業訂餐系統

Background

為了解決員工長期於廳內駐點餐廳用餐，而產生用餐疲勞的問題，公司計畫額外引入外部商家提供的外送服務，豐富員工的用餐選擇。然而，幾萬人的公司若手動管理外部訂單，將面臨：餐廳資訊混亂、份數統計出錯、領餐現場失控等管理困境。本系統旨在提供一個自動化平台，讓外部商家能自主管理菜單與供應量，並讓員工得以提前預購餐點。

Target Audience

- 員工：厭倦餐廳、想提前規劃未來數天午餐，並在指定地點領取外送美食的人
- 合作商家：能自行更新菜單、設定每日最大供應份數（限量），並根據總單據進行備餐與配送的人
- 福委會管理員：負責審核外部商家入駐，並透過系統自動對帳扣款，不需介入每日訂單細節

Interview Synthesis

核心訪談結論（作為 PRD 需求來源）

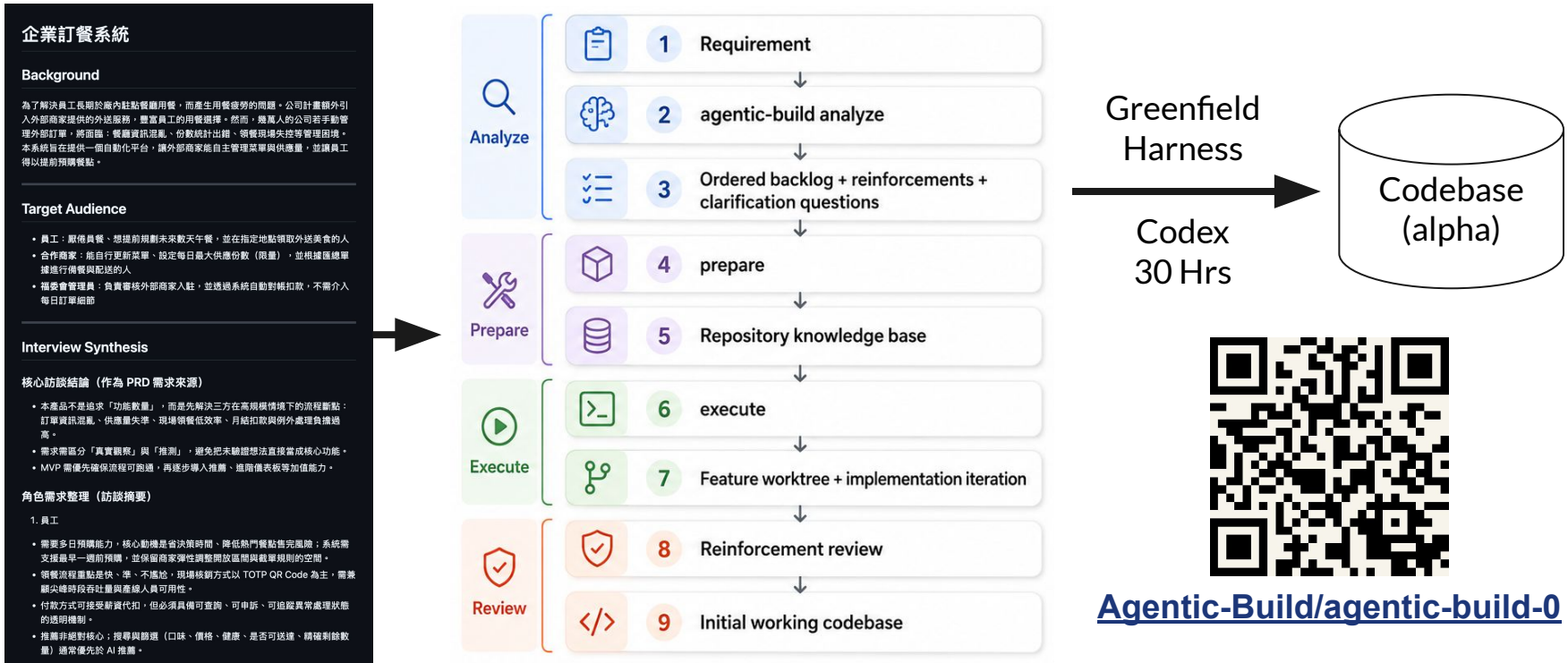
- 本產品不是追求「功能數量」，而是先解決三方在高規模情境下的流程斷點：訂單資訊混亂、供應量失準、現場領餐低效率、月結扣款與例外處理負擔過高。
- 需求需區分「真實觀察」與「推測」，避免把未驗證想法直接當成核心功能。
- MVP 需優先確保流程可跑通，再逐步導入推薦、進階儀表板等加值能力。

角色需求整理（訪談摘要）

1. 員工

- 需要多日預購能力，核心動機是省決策時間、降低熱門餐點售完風險；系統需支援最早一週前預購，並保留商家彈性調整開放區間與載單規則的空間。
- 領餐流程重點是快、準、不尷尬，現場核銷方式以 TOTP QR Code 為主，需兼顧尖峰時段吞吐量與產線人員可用性。
- 付款方式可接受薪資代扣，但必須具備可查詢、可申訴、可追蹤異常處理狀態的透明機制。
- 推薦非絕對核心；搜尋與篩選（口味、價格、健康、是否可送達、精確剩餘數量）通常優先於 AI 推薦。

Greenfield: agentic-build as the harness



Unstructured inputs > Structured requirements > Implementation specs > Greenfield Codebase > BOIL

We follow:

Harness

Engineering

February 11, 2026 Engineering

Harness engineering: leveraging Codex in an agent-first world

By Ryan Lopopolo, Member of the Technical Staff



Listen to article

17:57



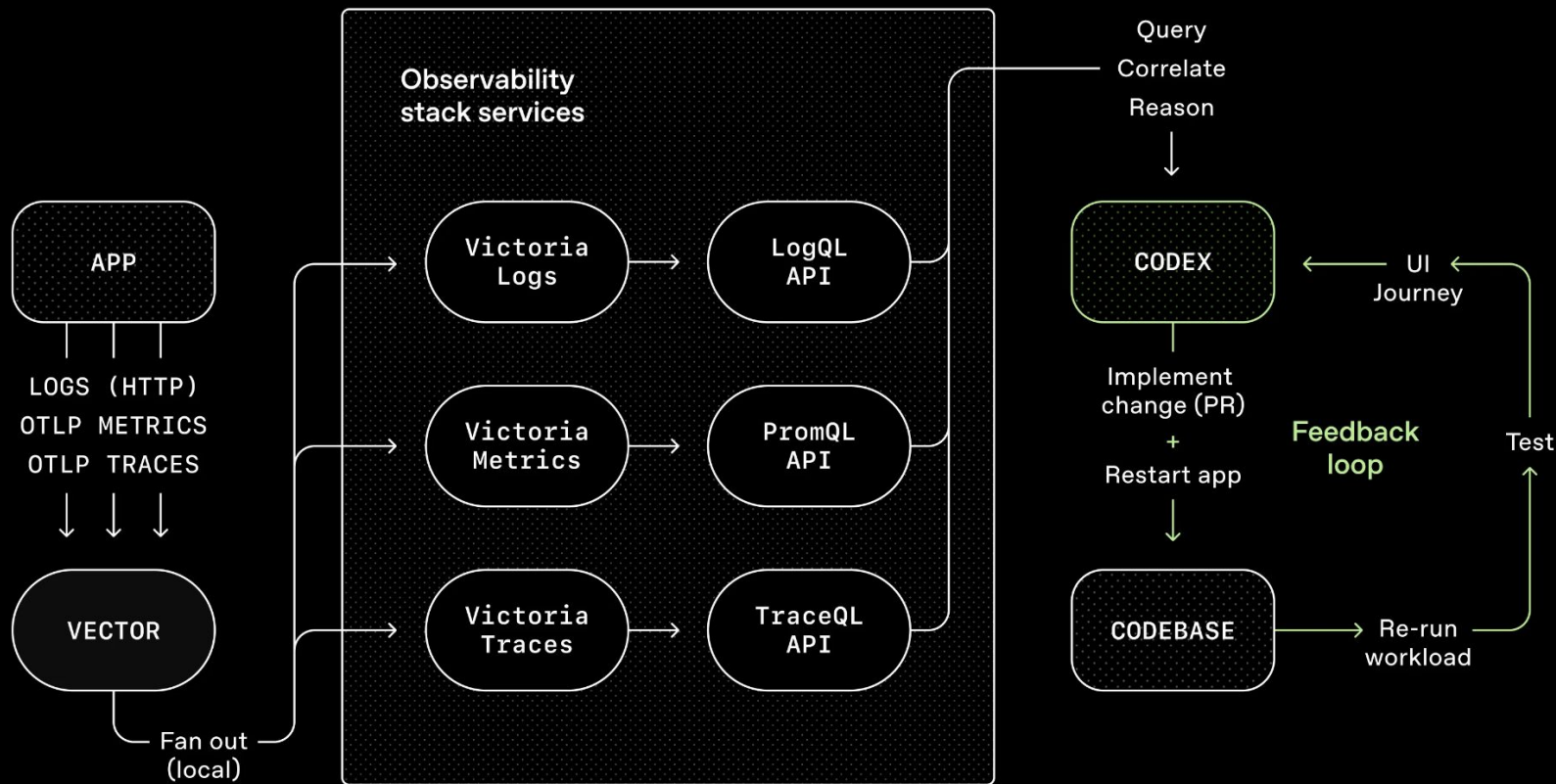
Share

Over the past five months, our team has been running an experiment: building and shipping an internal beta of a software product with **0 lines of manually-written code**.

The product has internal daily users and external alpha testers. It ships, deploys, breaks, and gets fixed. What's different is that every line of code—application logic, tests, CI configuration, documentation, observability, and internal tooling—has been written by Codex. We estimate that we built this in about 1/10th the time it would have taken to write the code by hand.

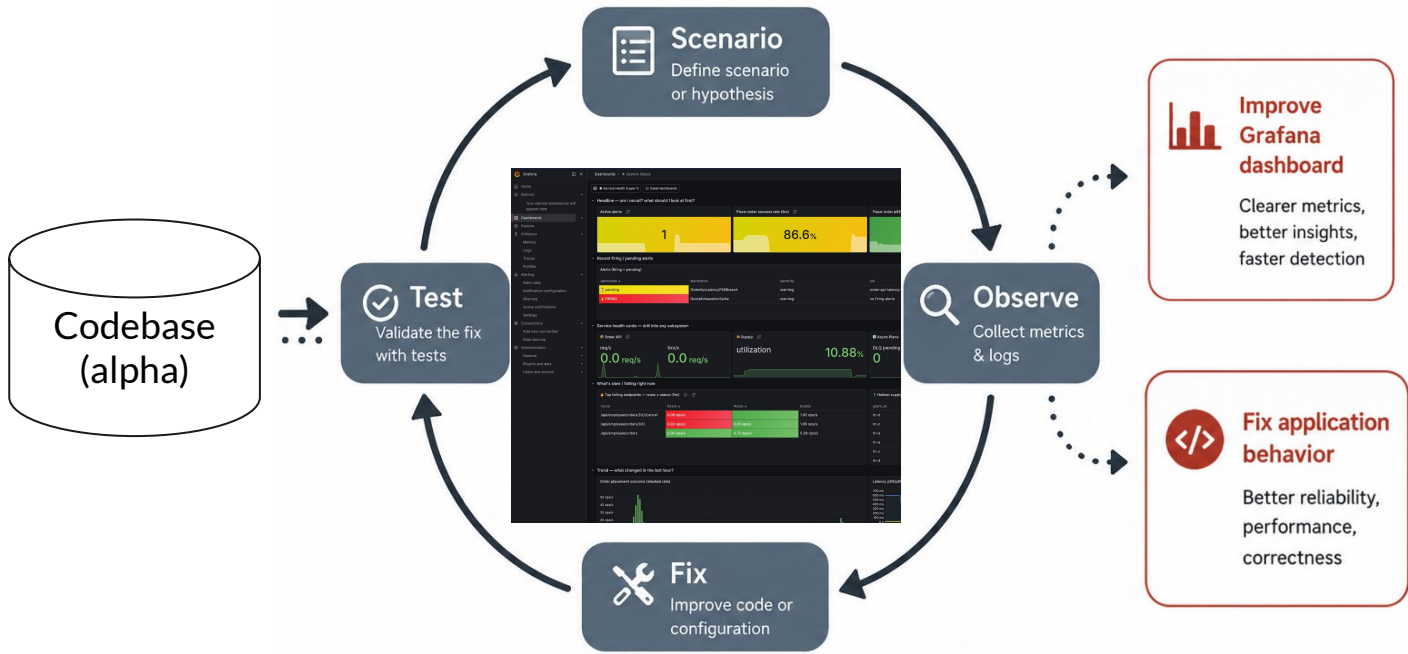
Humans steer. Agents execute.

Giving Codex a full observability stack



BOIL: Behavior & Observability Improvement Loop

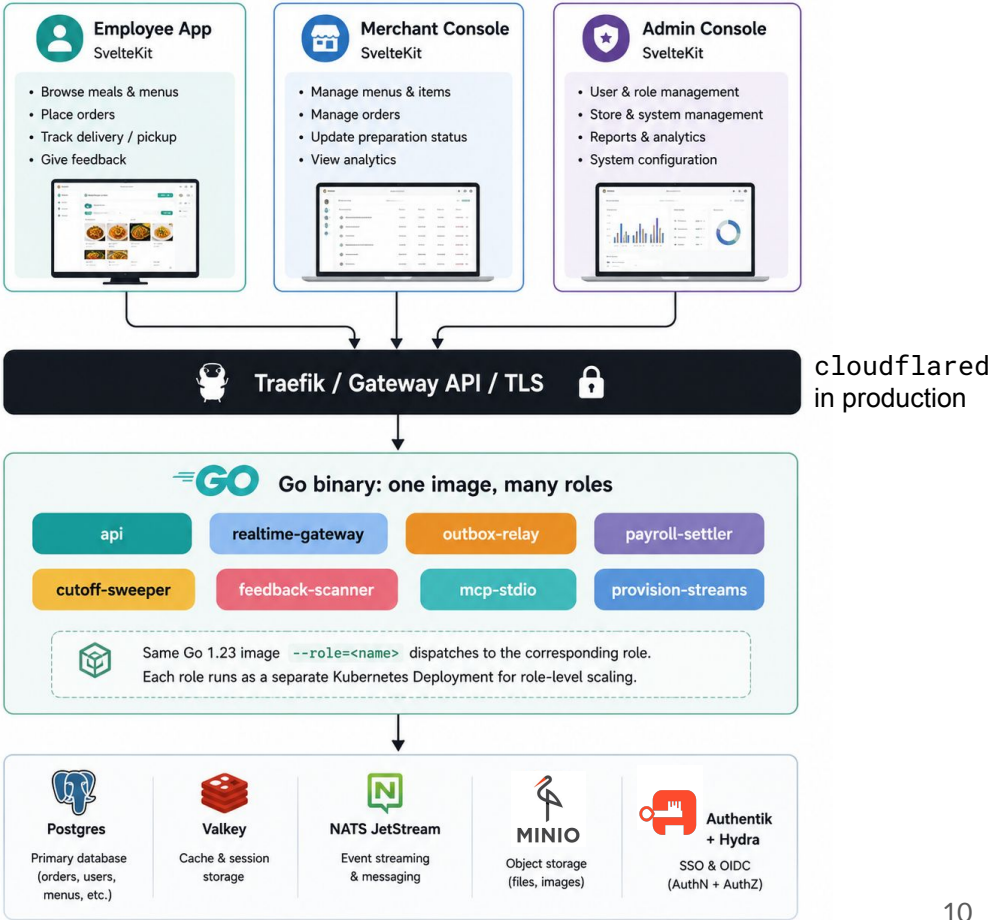
SOFT: Scenario → Observe → Fix → Test



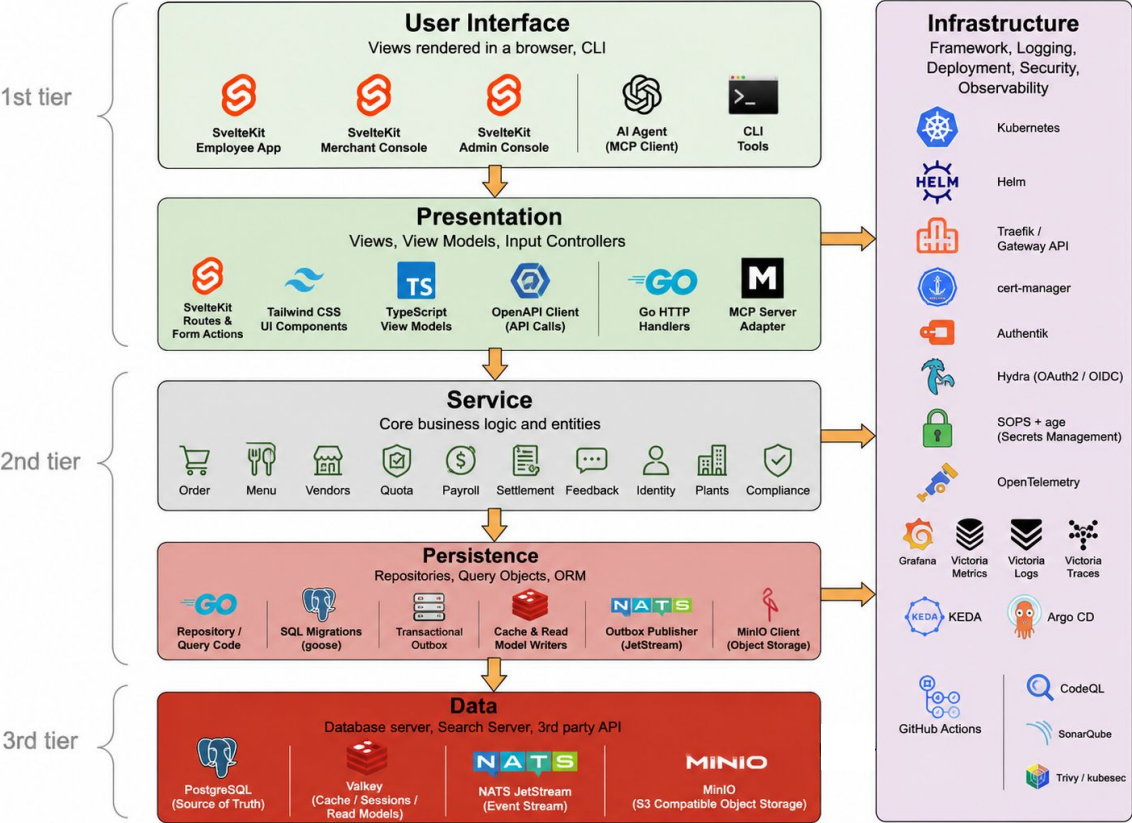
Architecture

Architecture:

3 apps + Go modular monolith

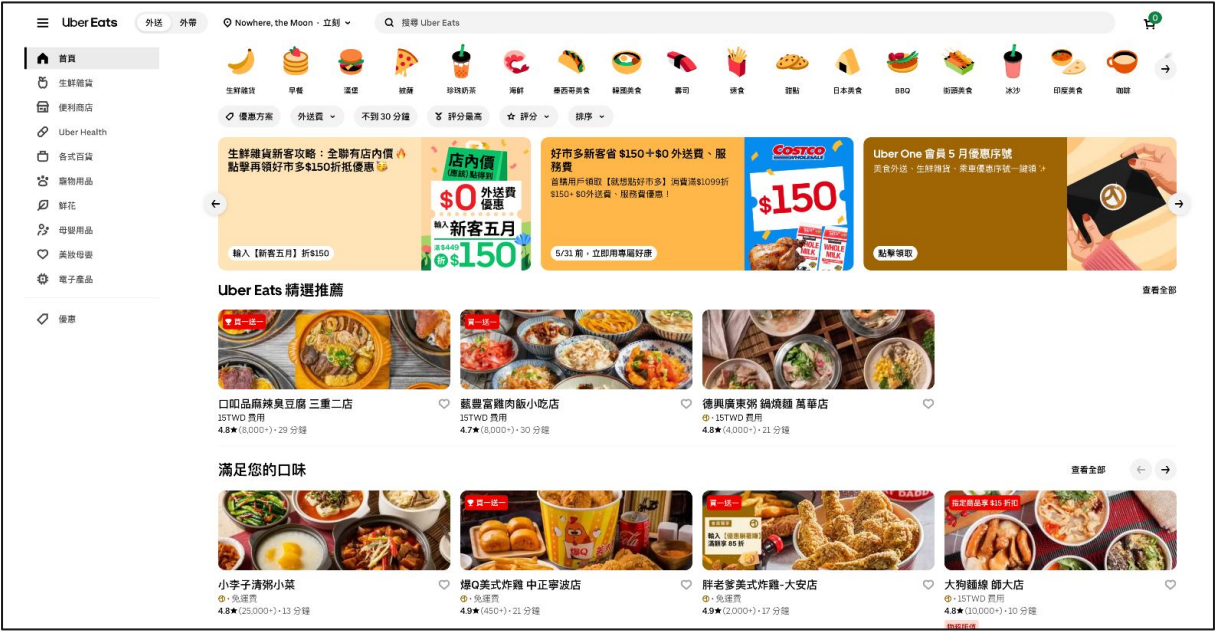


architecture mapping



Design

Why not just Uber Eats?



Why not just Uber Eats?

Uber Eats-like	T-Bite enterprise catering
個人即時點餐	企業預先訂餐
通常單店訂單	一餐可跨多店合併
送到個人	集中送到各廠取餐點
個人付款	福委會企業治理、對帳
訂單追蹤	利用掃描 QR Code 追蹤領餐狀態

Baseline: all checked

Target Audience

- 員工：厭倦員餐、想提前規劃一週午餐，並在指定地點領取外送美食的人。
- 合作商家：能自行更新菜單、設定每日最大供應份數（限量），並根據匯總單據進行備餐與配送的人。
- 福委會管理員：負責審核外部商家入駐，並透過系統自動對帳扣款，不需介入每日訂單細節。

System Requirement

- 商家自助管理後台：提供商家管理菜單、價格、照片與每日供應量的自動化介面。
- 員工預購與管理系統：員工能進行未來 7 天的預購，並查看訂單紀錄。
- 領餐與分流機制：系統自動產生標準化配送標籤供商家使用，並透過員工識別證實現快速領餐確認。
- 餐點推薦：整合外部環境 (如天氣) 與內部數據 (如銷量、營養)，提供多樣化的推薦機制。學生可自由設計呈現方式 (如搜尋優化、專區推薦)，並自主決定採用 AI 模型或 Rule-based 的方式。

Advanced Requirement

- 自動化結帳流程：每月自動匯出商家應收帳款報表，與公司現有薪資扣款系統整合。
- 福委會管理後台：提供福委會人員審核新商家加入的機制，並監控整體訂餐數據與營運狀況。
- 多廠區商家管理：福委會可以在後台設定商家所能服務的廠區。員工訂餐時，系統會自動根據他的廠區顯示專屬菜單，避免點到無法送達的店家。

Baseline: 3 apps

員工

商家

福委會

T-Bite
CORPORATE CATERING

HC 12A 1F · 今天

搜尋餐廳或餐點...

掃描領餐

E

今日首頁

我的訂單

掃描領餐

我的常點

我的寄訴

申訴

PRO TIP
前一日 17:00 前還可改單
請訂餐仍可至「我的訂單」裡改動、取餐點或取消。

推薦你今天
同事熱門 X 你的常用商家

2026 / 05 / 27 · 週三

哈囉，E2E Employee

今天的午餐已就緒，可點餐

選擇餐日 · 可預訂未來 7 天

27
今天

今天的訂單
\$50

再點一次
你最近的訂單，一鍵重新

阿城炙燒便當
三杯香脆豬便當
\$280

明天的菜色
今日 17:00 截單，之後將無法加入或下單。

三杯香脆豬便當
九層塔與蒜泥油膏蒸、蒜香煎香。

古早味排骨便當
厚切里肌與柴魚提味，搭配油蔥與時蔬。

會議便當組合
五珍王煎豬腿便當，適合團體訂。

椒麻雞便當

T-Bite
CORPORATE CATERING

商家後台 · E2E Merchant

登出

E2

備餐儀表板

備餐看板

菜單管理

客訴收件匣

對帳

合規自查

營運設定

2026 / 05 / 27 · 週三 · 今日營運

今日備餐儀表板

2026 / 05 / 27 · 週三 · 合計 1447 份

今日份數
1447 份

SCHEDULE · 排程菜單

未來 7 天排餐

為每一天預先安排菜色與上層份數。所有菜色從「排餐」中選擇。

今天
15 週三 · 1420/1447

明天
15 週四 · 1512/1519

今天 17:00 截單

明天的菜色
今日 17:00 截單，之後將無法加入或下單。

品項

三杯香脆豬便當
九層塔與蒜泥油膏蒸、蒜香煎香。

古早味排骨便當
厚切里肌與柴魚提味，搭配油蔥與時蔬。

會議便當組合
五珍王煎豬腿便當，適合團體訂。

椒麻雞便當

T-Bite
CORPORATE CATERING

福委會後台 · 管理員

登出

福

治理總覽

商家管理

廠區登錄

月結

商家結算

升級客訴

告警

稽核紀錄

商家管理

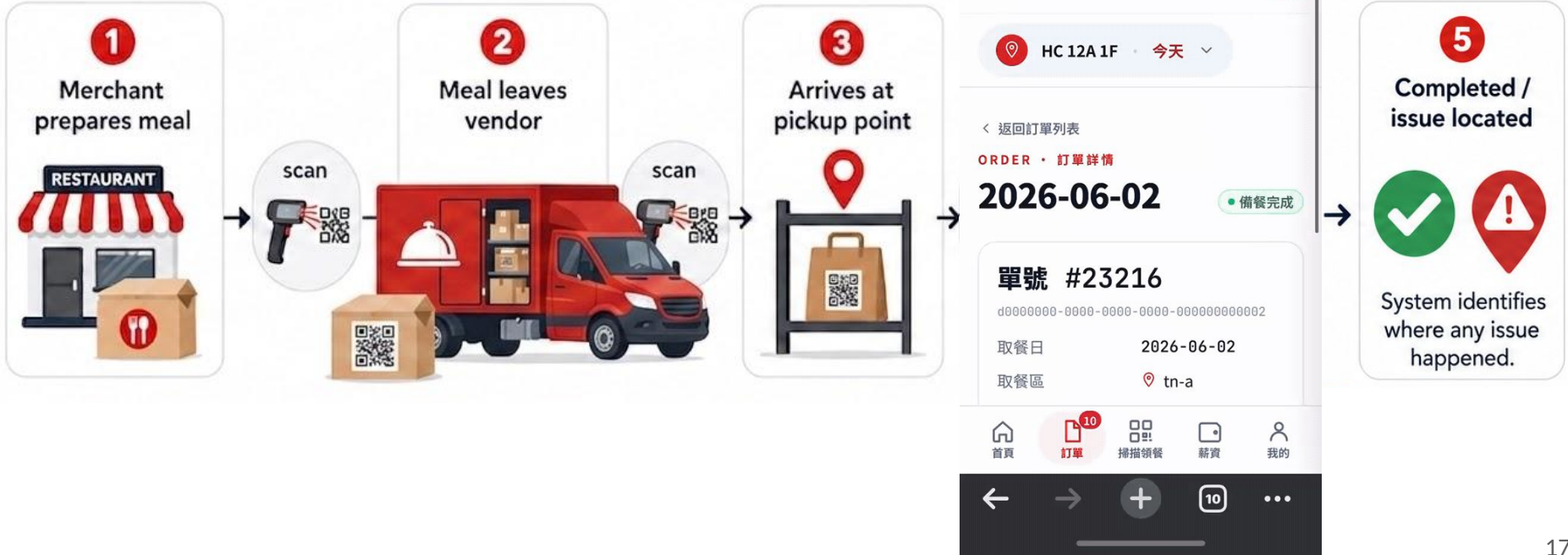
入駐審核、廠區配對與停復權 · 操作會自動寫入稽核日誌

全部 待審 已核准 停權中

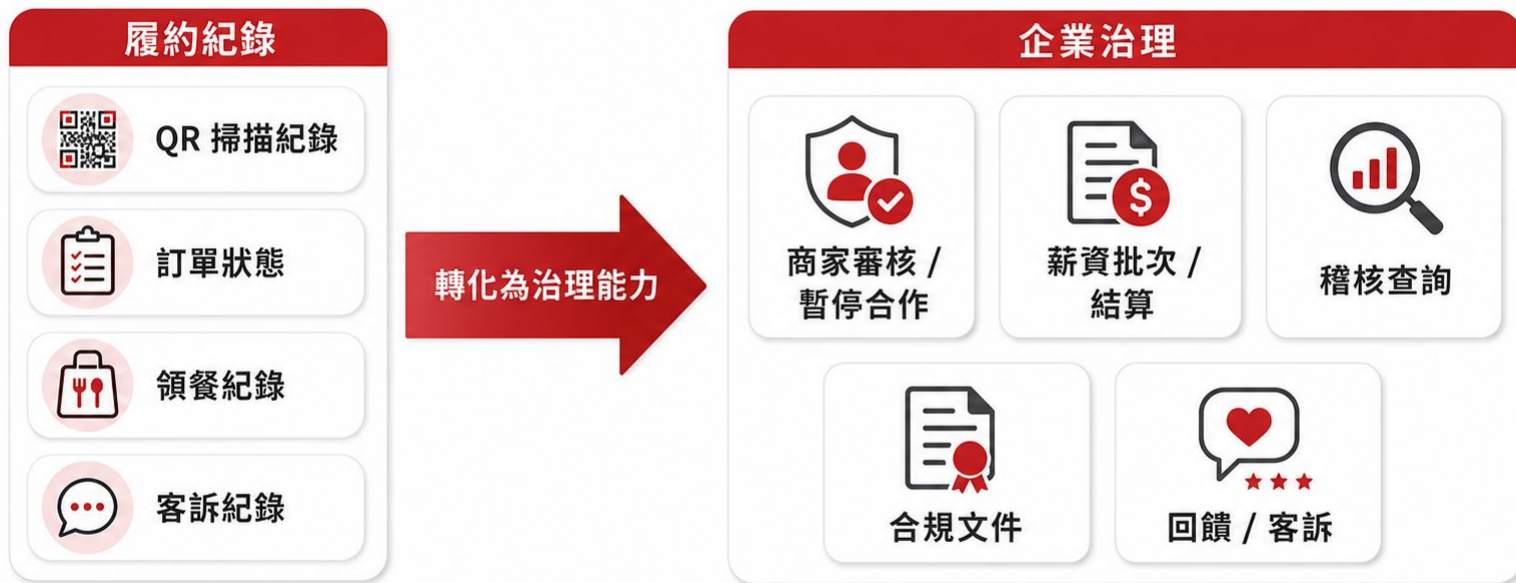
新增 Pending 商家

商家	EMAIL	狀態	服務廠區	詳細
最強小木屋	wood@tbite.test	已核准	hc-12a-3f · tc-15a-3f · tn-a	詳細
Takala 小石屋	takala@tbite.test	已核准	tn-a	詳細
Takala 小木屋	e2e-merchant@tbite.test	停權	—	詳細

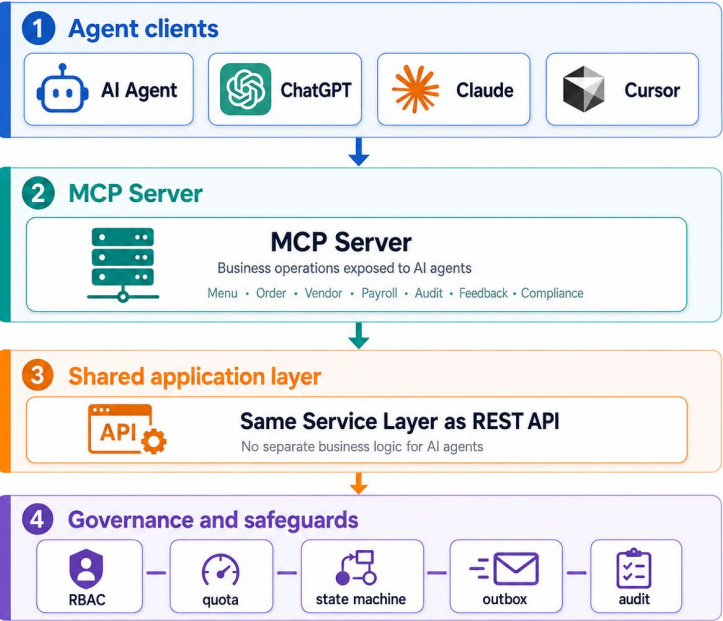
Feature 1: QR code Pipeline



Feature 2: Enterprise governance

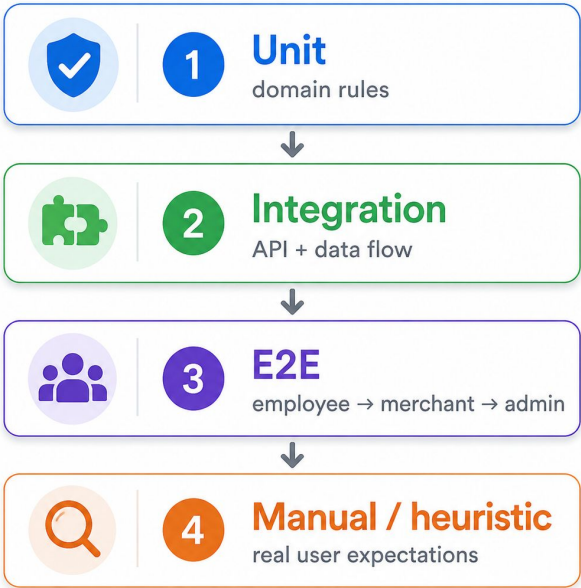


Feature 3: MCP

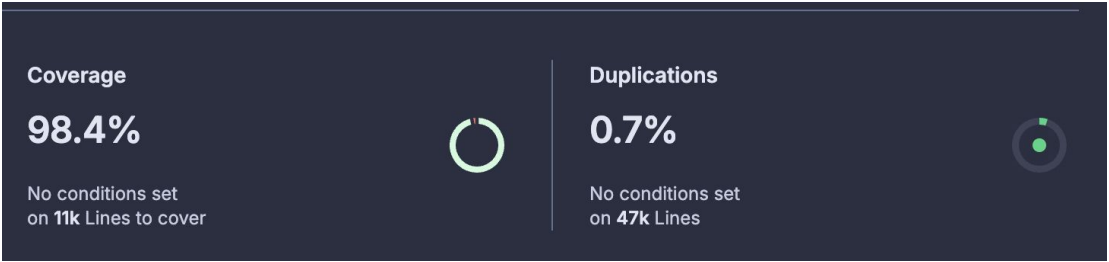


Testing

Validation layers and quality gates



- Using go test, vitest, playwright, testify etc...
- Include unit, integration, e2e test on codebase.
- Coverage 98.4% and Duplications only 0.7%



SonarQube

Scale validation



Test Environment:

OrbStack on M5 Pro.

400% CPU & 9G RAM used.

applied demo

TSMC-style demo data:

- 50,000 employees
- 10 merchants
- 150 menu items
- 19 pickup points

<https://tbite-admin.nycu.ai/>

stress benchmark

- 100,000 employees
- 200 merchants
- 200 pickup points
- 1,000,000 backend API requests
- 0 failed
- 571.7 people/s
- 5717.3 backend Request Per Sec.

Chaos pass criteria

- Delete a pod
- Controller returns Ready < 3 min
- Menu browsing remains available
- Order succeeds or expected 409
- No sustained 5xx
- Grafana shows fault + recovery
- No manual DB repair

Human-AI Heuristic Evaluation on Frontend

1. AI Functional Check

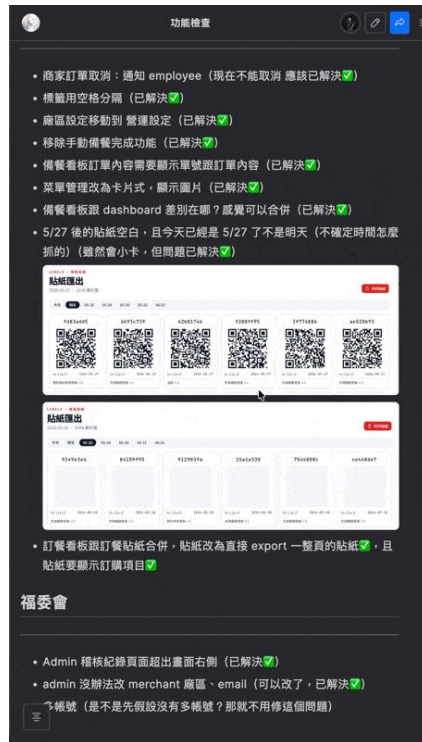
Claude Design checks whether buttons, pages, and flows are executable.

2. Human UX Review

We manually inspected whether the UI matches real users' expectations.

3. Frontend Refinement

Because agent-generated frontend was weaker, we used human review to find layout, wording, and flow issues.



Cybersecurity Matters Too

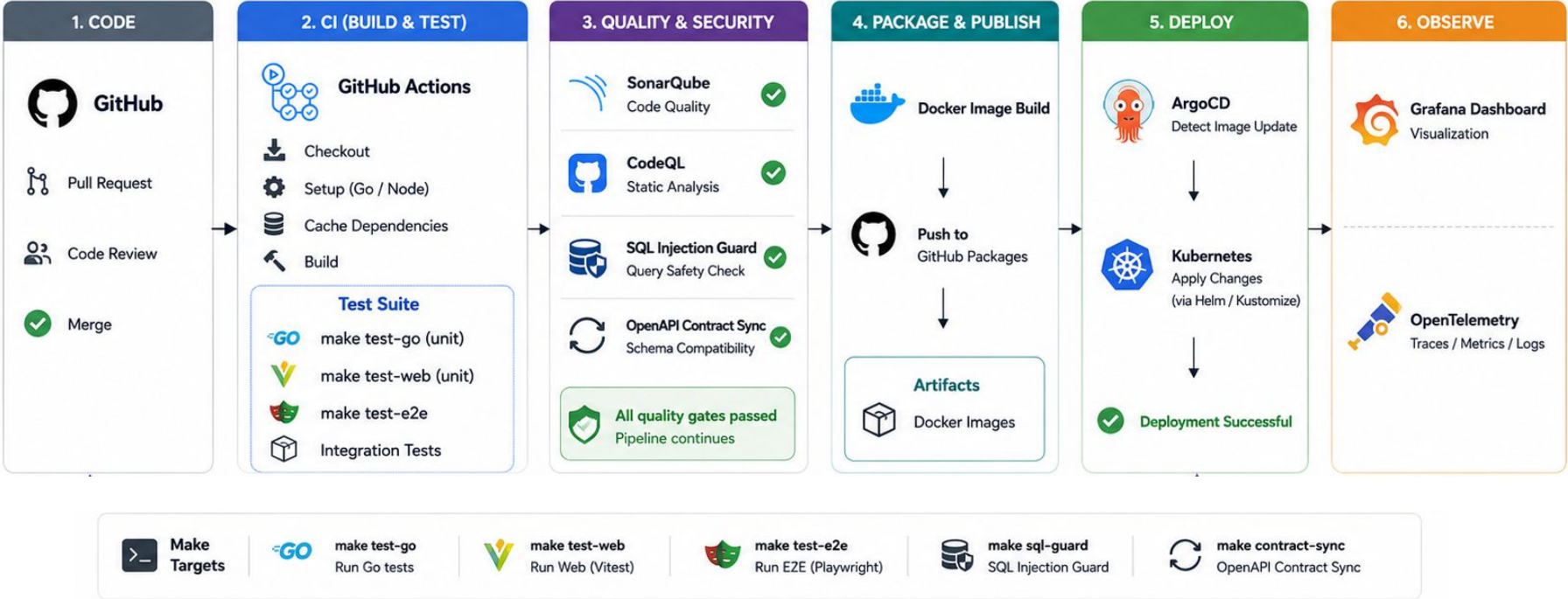
- Enterprise SSO + OAuth/OIDC
- CI/CD scan: CodeQL (CVE) + SonarQube (code smell)

The screenshot displays the 'Summary' page of the 'corporate-catering-system' repository on the Agentic Build platform. The interface is dark-themed. At the top, it shows the repository name and a 'Summary' tab. Below this, there's a 'main' branch selector with a green checkmark icon. A 'Public' badge is visible, along with '30k Lines of Code' and 'Last analysis 16 minutes ago'. A 'View on GitHub' button is on the right. The main content area has two tabs: 'New Code' and 'Overall Code'. Under 'Overall Code', there are three sections: 'Security', 'Reliability', and 'Maintainability'. Each section shows '0 Open issues' and a green circle with the letter 'A', indicating a high quality score.

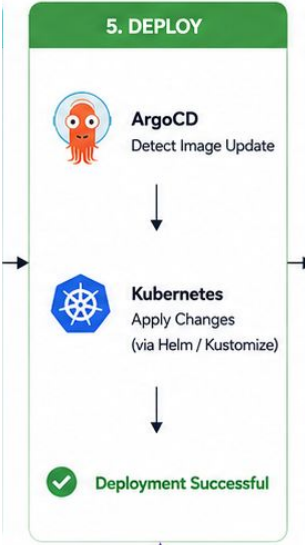
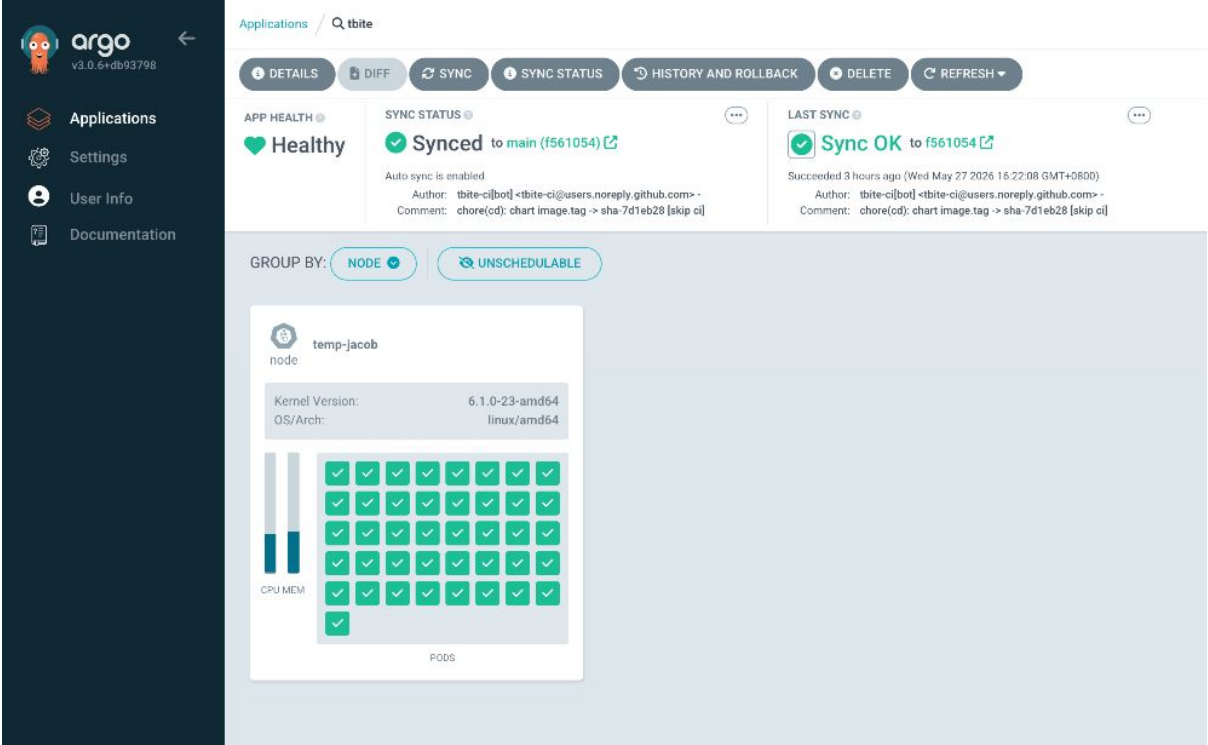
Category	Open Issues	Score
Security	0	A
Reliability	0	A
Maintainability	0	A

Operations

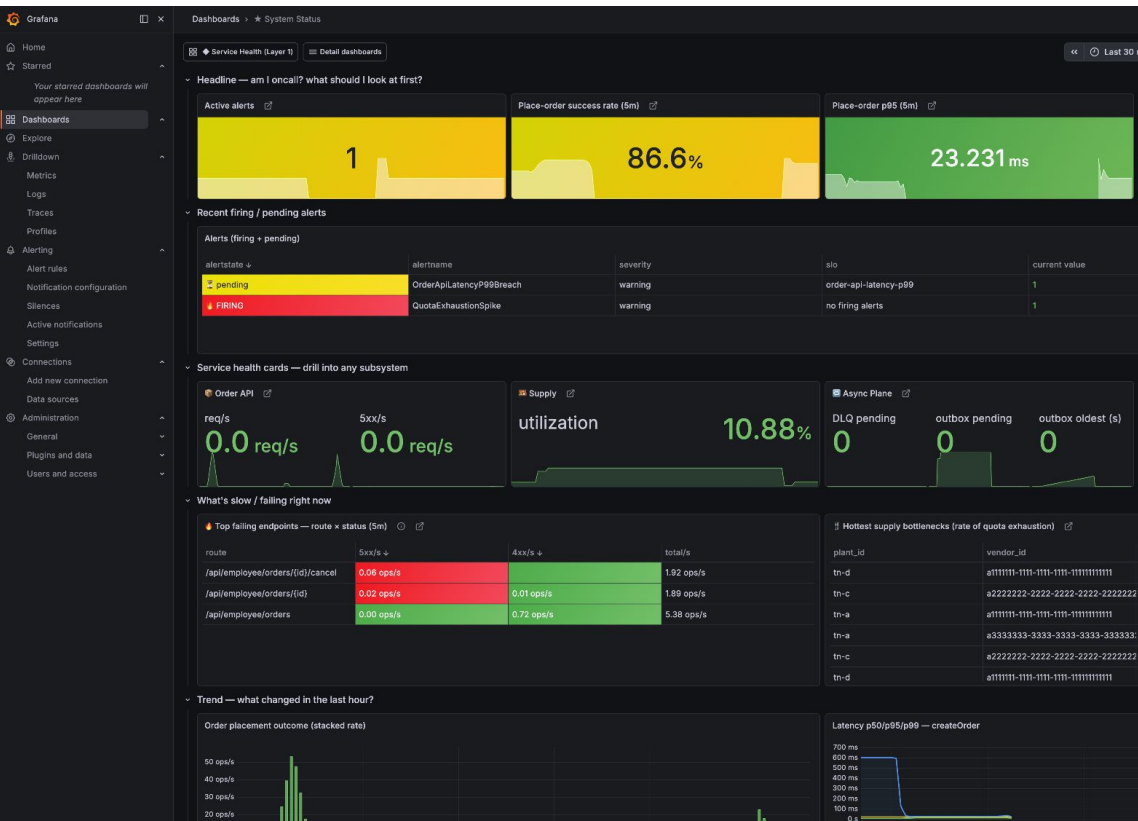
CI/CD



DevOps - argoCD

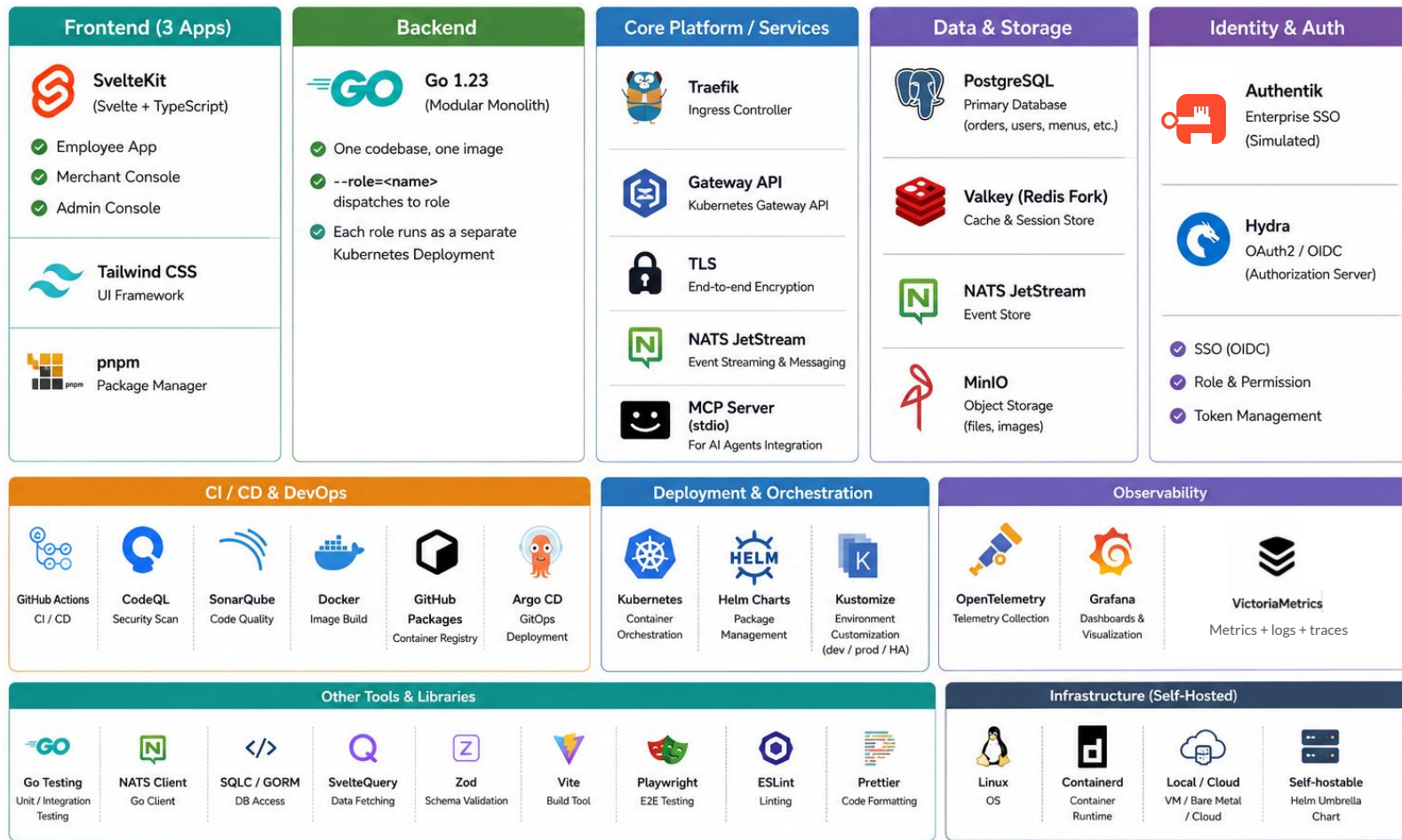


Observability - Grafana



Conclusion

System Architecture



158,914 LoC

Total in Codebase @d567337

o LoC

Written by Hand

3,600,000,000 tokens

To ignite a working codebase from scratch by GPT 5.3 Codex.

10,000,000,000 tokens

To complete the all project, with days of auto improvement and adversarial testing
with Claude Opus 4.7/4.8 and GPT-5.4/5.5.

\$30 per hour

That's \$260k per year for an engineer without sleep!

Thanks.

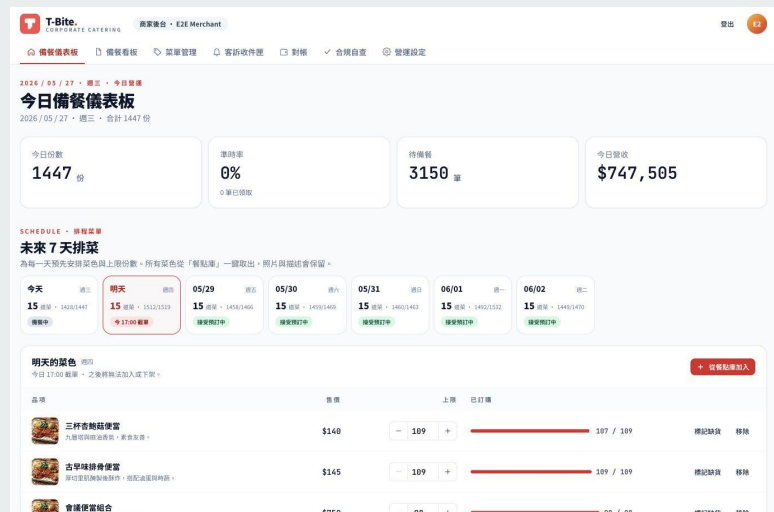
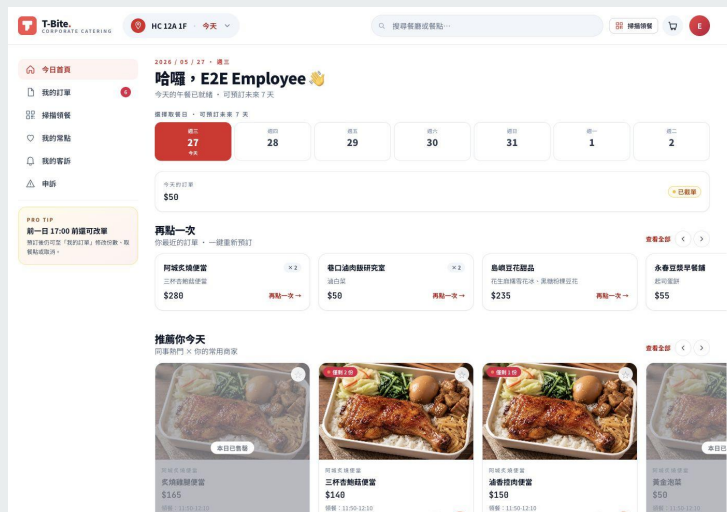
AI-Agent-Driven Enterprise Catering Platform:

From Greenfield Generation to Self-Hostable Kubernetes Validation



T-Bite.
CORPORATE CATERING

Team 10
林振可 Jacob
王重鈞
Takala
邱可茵 DW
洪軾凱 Sky



Appendix: 後續評審提問

報告 / QA 錄音：

https://web.plaud.ai/s/pub_ffc69912-cb15-412f-9255-ef00a5ef2d11::sTq-v76964vl2luWX5VnTkVPe2m3Tf5_UJgFcGHxhlqlalEo94HPNSTAIUyg8L9-vmut8lO1vinNMIC

Q1: 如果系統遇到狀況，是交給 IT 嗎？你們有監控嗎？ 監控發現問題後，誰會去處理？

我們有透過 Grafana 和 observability stack 監控系統狀態。

出問題時，第一線會先交給 agent SREs 做分類與初步判斷。

如果是比較麻煩、需要人類判斷的問題，才會回到我們團隊處理。

Q2: 你們有真的進去看系統發生什麼嗎？如果發現行為和預期不同，會怎麼處理？

在開發過程中，如果我們發現前端行為不是預期的樣子，我們通常不是直接去看資料庫。

我們會重新描述「我們需要的做法是什麼」，再讓 agent 根據這個行為需求去修改。

Q3: 有沒有比較印象深刻的差異？ Agent 哪些地方比較容易卡住？

Agent 過不去的地方大多是在前端。後端其實幾乎沒有遇到太多問題，但前端問題很多。

我們花了很多時間處理：

- 使用者預期的操作方式
- accessibility 問題
- 明顯的前端互動錯誤
- 功能語意不清楚造成的錯誤 產物

如果是整體上，有一個例子是去除多出來的功能：

系統原本做出了員工端不需要看的報表，但實際上這類資訊不應該給員工端使用。所以我們後來不是加功能，而是把這個不必要的功能砍掉。

Q4: 前端和後端的 AI 效果差在哪？ 這些問題是功能做不到，還是描述不清楚？

很多問題來自功能層面的描述不夠清楚。

前端的部分，即使有 AI 協助，效果仍然有限，需要人工驗證。

但在後端以及相關系統設計上，AI 產出的結果相對穩定，也能在壓力測試中達到穩定執行狀態。

Q5: Database schema 在 optimization loop 裡會怎麼處理？會不會一直變？

Database schema 會在 optimization loop 裡一直變動。

例如 scenario 可能要求 Service Level Agreement 或 performance, agent 會為了效能或需求去調整 schema。

我們認為這些 schema 變動在當下可以幫助系統達到最終目標, 但缺點是 schema 可能變動過於頻繁。

實際部署後, 如果要避免這種快速變動, 可能會變成另一個 requirement, 也就是需要限制 schema 變動不要太劇烈。